

Status Finished

Started Tuesday, 14 October 2025, 9:40 PM

Completed Wednesday, 15 October 2025, 10:26 AM

Duration 12 hours 45 mins

Marks 24/29

Grade 83 out of 100

Question 1

Incorrect

Mark 0 out of 1

Which of the following isn't a good way to handle missing values when they are in rows that represent valid cases?

- ☒ a. back fill their values ❌
- ☐ b. drop their rows
- ☐ c. forward fill their values
- ☐ d. use interpolation to fill their values

Question 2

Correct

Mark 1 out of 1

The **firesDataFrame**

	fire_name	fire_year	state	discovery_date	contain_date	acres_burned
16	POWER	2004	CA	2004-10-06	2004-10-21	16823.0
17	FREDS	2004	CA	2004-10-13	2004-10-17	7700.0
25	BACHELOR	2004	NM	2004-07-20	2004-07-20	10.0
37	HOWARD GAP	2005	NC	2005-01-27	2005-01-28	50.3
39	AUSTIN CREEK	2005	NC	2005-02-12	2005-02-13	125.0
...	...	...	...	...	...	...
1880370	WITCHER (4-6)	2010	CA	2010-07-26	2010-07-27	81.0
1880377	HAPPY	2015	CA	2015-07-29	2015-08-03	30.0
1880387	SODA	2015	CA	2015-07-04	2015-07-04	24.0
1880399	GILMAN	2009	CA	2009-05-03	2009-05-03	73.0
1880411	TERRA	2009	CA	2009-06-19	2009-06-19	23.0

96239 rows × 6 columns

Refer to the **firesDataFrame**. This code adds a column to the DataFrame for the number of days that a fire burned:

- ☐ a. `fires['burning_days'] = \`
`fires.contain_date.dt.days - fires.discovery_date.dt.days`
- ☒ b. `fires['burning_days'] = \` ✓
`(fires.contain_date - fires.discovery_date).dt.days`
- ☐ c. `fires['burning_days'] = \`
`(fires.discovery_date - fires.contain_date).dt.days`
- ☐ d. `fires['burning_days'] = \`
`fires.discovery_date - fires.contain_date`

Question 3

Correct

Mark 1 out of 1

The **firesDataFrame**

	fire_name	fire_year	state	discovery_date	contain_date	acres_burned
16	POWER	2004	CA	2004-10-06	2004-10-21	16823.0
17	FREDS	2004	CA	2004-10-13	2004-10-17	7700.0
25	BACHELOR	2004	NM	2004-07-20	2004-07-20	10.0
37	HOWARD GAP	2005	NC	2005-01-27	2005-01-28	50.3
39	AUSTIN CREEK	2005	NC	2005-02-12	2005-02-13	125.0
...	...	...	...	...	...	...
1880370	WITCHER (4-6)	2010	CA	2010-07-26	2010-07-27	81.0
1880377	HAPPY	2015	CA	2015-07-29	2015-08-03	30.0
1880387	SODA	2015	CA	2015-07-04	2015-07-04	24.0
1880399	GILMAN	2009	CA	2009-05-03	2009-05-03	73.0
1880411	TERRA	2009	CA	2009-06-19	2009-06-19	23.0

96239 rows × 6 columns

Refer to the **firesDataFrame**. This code adds a summary column to the DataFrame that contains the average acres burned by year:

- ☐ a. `fires['acres_yearly_mean'] = \`
`fires.acres_burned.groupby('fire_year').transform('mean')`
- ☐ b. `fires['acres_yearly_mean'] = \`
`fires.groupby('fire_year').acres_burned.apply('mean')`
- ☒ c. `fires['acres_yearly_mean'] = \`
`fires.groupby('fire_year').acres_burned.transform('mean')` ✓
- ☐ d. `fires['acres_yearly_mean'] = fires.groupby('fire_year').mean()`

Question 4

Correct

Mark 1 out of 1

When a column has the category data type, the data in the column

- ☐ a. requires more memory
- ☒ b. consists of integers that point to text entries in a table ✓
- ☐ c. consist of strings that point to a table of category counts
- ☐ d. consists of string labels for the categories

Question 5

Correct

Mark 1 out of 1

To fix the SettingWithCopyWarning, you can

- ☐ a. use a reference instead of a copy
- ☐ b. use the `.loc[]` accessor to create a new DataFrame
- ☒ c. use the `copy()` method to copy the original DataFrame ✓

Question 6

Incorrect

Mark 0 out of 1

Perhaps the best way to smooth the effect of outliers in your line plots is to

- ☒ a. cap the values of the outliers ❌
- ☐ b. use interpolation to assign values to the outliers
- ☐ c. assign the mean value of the column to the outliers
- ☐ d. drop the rows that are outliers

Question 7

Correct

Mark 1 out of 1

An inner join

- ☐ a. combines all of the rows in the first DataFrame with the rows in the second DataFrame that have matching indexes
- ☐ b. combines all of the rows in both DataFrames
- ☒ c. combines only the rows in the first and second DataFrames that have matching indexes ✅

Question 8

Correct

Mark 1 out of 1

The **firesDataFrame**

	fire_name	fire_year	state	discovery_date	contain_date	acres_burned
16	POWER	2004	CA	2004-10-06	2004-10-21	16823.0
17	FREDS	2004	CA	2004-10-13	2004-10-17	7700.0
25	BACHELOR	2004	NM	2004-07-20	2004-07-20	10.0
37	HOWARD GAP	2005	NC	2005-01-27	2005-01-28	50.3
39	AUSTIN CREEK	2005	NC	2005-02-12	2005-02-13	125.0
...	...	...	...	...	...	...
1880370	WITCHER (4-6)	2010	CA	2010-07-26	2010-07-27	81.0
1880377	HAPPY	2015	CA	2015-07-29	2015-08-03	30.0
1880387	SODA	2015	CA	2015-07-04	2015-07-04	24.0
1880399	GILMAN	2009	CA	2009-05-03	2009-05-03	73.0
1880411	TERRA	2009	CA	2009-06-19	2009-06-19	23.0
96239 rows × 6 columns						

Refer to the **firesDataFrame**. This code adds a new column for the discovery month:

- ☐ a. `fires['discovery_month'] = fires.discovery_date.month`
- ☐ b. `fires['discovery_month'] = fires.discovery_date.month.dt`
- ☐ c. `fires['discovery_month'] = discovery_date.dt.month`
- ☒ d. `fires['discovery_month'] = fires.discovery_date.dt.month` ✅

Question 9

Correct

Mark 1 out of 1

The two common data type problems that occur when you import data into a DataFrame are

- ☐ a. datetime and category columns with the object data type
- ☐ b. numeric columns with the object type and datetime columns with the numeric type
- ☒ c. numeric and datetime columns with the object data type ✓
- ☐ d. numeric and category columns with the object data type

Question 10

Correct

Mark 1 out of 1

The two main methods for finding outliers are

- ☐ a. the mean() and plot() methods
- ☐ b. the plot() and max() methods
- ☒ c. the describe() and plot() methods ✓
- ☐ d. the mean() and max() methods

Question 11

Correct

Mark 1 out of 1

You should use the join() method when you want to

- ☒ a. combine two DataFrames based on their index ✓
- ☐ b. add the rows of one DataFrame to another DataFrame
- ☐ c. combine two columns in a DataFrame
- ☐ d. combine two DataFrames based on the data in one or more columns

Question 12

Correct

Mark 1 out of 1

The mortality DataFrame

	Year	Age_Group	Death_Rate
0	1900	1-4 Years	1983.8
1	1901	1-4 Years	1695.0
2	1902	1-4 Years	1655.7
3	1903	1-4 Years	1542.1
4	1904	1-4 Years	1591.5
...	...	...	...
471	2014	15-19 Years	45.5
472	2015	15-19 Years	48.3
473	2016	15-19 Years	51.2
474	2017	15-19 Years	51.5
475	2018	15-19 Years	49.2

476 rows × 3 columns

The mortality_mean DataFrame

	MeanDeathRate
0	383.22605
1	383.22605
2	383.22605
3	383.22605
4	383.22605
...	...
471	177.37479
472	177.37479
473	177.37479
474	177.37479
475	177.37479

476 rows × 1 columns

Refer to the **mortality DataFrame**. This code unstacks the data in the AgeGroup column by AgeGroup within Year::

- ☐ a. `mortality.unstack().set_index(['Age_Group', 'Year'])`
- ☒ b. `mortality.set_index(['Year', 'Age_Group']).unstack()` ✓
- ☐ c. `mortality.unstack(['Year', 'Age_Group'])`
- ☐ d. `mortality.set_index(['Age_Group', 'Year']).unstack()`

Question 13

Correct

Mark 1 out of 1

This statement forward fills missing values with a limit of two consecutive fills:

- ☐ a. `mortality = mortality.interpolate()`
- ☒ b. `mortality = mortality.ffill(limit=2)` ✓
- ☐ c. `mortality = mortality.ffill()`
- ☐ d. `mortality = mortality.interpolate(limit=2)`

Question 14

Correct

Mark 1 out of 1

The **firesDataFrame**

	fire_name	fire_year	state	discovery_date	contain_date	acres_burned
16	POWER	2004	CA	2004-10-06	2004-10-21	16823.0
17	FREDS	2004	CA	2004-10-13	2004-10-17	7700.0
25	BACHELOR	2004	NM	2004-07-20	2004-07-20	10.0
37	HOWARD GAP	2005	NC	2005-01-27	2005-01-28	50.3
39	AUSTIN CREEK	2005	NC	2005-02-12	2005-02-13	125.0
...	...	...	...	...	...	...
1880370	WITCHER (4-6)	2010	CA	2010-07-26	2010-07-27	81.0
1880377	HAPPY	2015	CA	2015-07-29	2015-08-03	30.0
1880387	SODA	2015	CA	2015-07-04	2015-07-04	24.0
1880399	GILMAN	2009	CA	2009-05-03	2009-05-03	73.0
1880411	TERRA	2009	CA	2009-06-19	2009-06-19	23.0

96239 rows × 6 columns

Refer to the **firesDataFrame**. This code uses a lambda expression to add a Boolean column to the DataFrame that indicates whether a state touches the Pacific Ocean:

- ☐ a. `states = ['CA','OR','WA','AK','HI']`
`fires['ocean_state'] = /`
`fires.state(lambda x: 1 if x == states else 0)`
- ☒ b. `states = ['CA','OR','WA','AK','HI']` ✓
`fires['ocean_state'] = /`
`fires.apply(lambda x: 1 if x.state in states else 0, axis=1)`
- ☐ c. `states = ['CA','OR','WA','AK','HI']`
`fires['ocean_state'] = /`
`fires.apply(lambda x: 1 if x.state in states else 0)`
- ☐ d. `states = ['CA','OR','WA','AK','HI']`
`fires['ocean_state'] = /`
`fires.state(lambda x: 1 if x in states else 0, axis=1)`

Question 15

Correct

Mark 1 out of 1

This statement displays all rows that have one or more missing values:

- ☐ a. `mortality[mortality.notnull().any(axis=1)]`
- ☐ b. `mortality[mortality.notnull()]`
- ☐ c. `mortality[mortality.isnull()]`
- ☒ d. `mortality[mortality.isnull().any(axis=1)]` ✓

Question 16

Correct

Mark 1 out of 1

The mortality DataFrame

	Year	Age_Group	Death_Rate
0	1900	1-4 Years	1983.8
1	1901	1-4 Years	1695.0
2	1902	1-4 Years	1655.7
3	1903	1-4 Years	1542.1
4	1904	1-4 Years	1591.5
...	...	...	...
471	2014	15-19 Years	45.5
472	2015	15-19 Years	48.3
473	2016	15-19 Years	51.2
474	2017	15-19 Years	51.5
475	2018	15-19 Years	49.2

476 rows × 3 columns

The mortality_mean DataFrame

	MeanDeathRate
0	383.22605
1	383.22605
2	383.22605
3	383.22605
4	383.22605
...	...
471	177.37479
472	177.37479
473	177.37479
474	177.37479
475	177.37479

476 rows × 1 columns

Refer to the **mortality** and **mortality_mean** DataFrames. This code combines the data in the two DataFrames:

- ☐ a. `mortality.concat(mortality_mean)`
- ☒ b. `mortality.join(mortality_mean)` ✓
- ☐ c. `pd.concat([mortality_data, mortality_mean])`
- ☐ d. `mortality.merge(mortality_mean)`

Question 17

Correct

Mark 1 out of 1

To get the count for each unique value in a column, you can use the

- ☐ a. `unique()` method
- ☒ b. `value_counts()` methods ✓
- ☐ c. `nunique()` method
- ☐ d. `unique()` and `value_counts()` methods

Question 18

Correct

Mark 1 out of 1

To find columns that have as many unique values as rows, you can use the

- ☐ a. `unique()` method
- ☒ b. `info()` and `nunique()` methods ✓
- ☐ c. `info()` and `unique()` methods
- ☐ d. `nunique()` method

Question 19

Correct

Mark 1 out of 1

One way to drop unnecessary rows is to

- ☐ a. use the query() method to select the rows you want and drop all the other rows
- ☒ b. use the query() method to select the rows you want and assign them to the DataFrame ✓
- ☐ c. use the query() method to find the rows and the drop_rows() method to drop them
- ☐ d. use the query() method to find the rows and the drop() method to drop them

Question 20

Correct

Mark 1 out of 1

To find columns that have just one unique value, you can use the

- ☐ a. info() and nunique() methods
- ☐ b. info() and unique() methods
- ☐ c. unique() method
- ☒ d. nunique() method ✓

Question 21

Correct

Mark 1 out of 1

A left join

- ☐ a. combines all of the rows in both DataFrames
- ☒ b. combines all of the rows in the first DataFrame with the rows in the second DataFrame that have matching indexes ✓
- ☐ c. combines only the rows in the first and second DataFrames that have matching indexes

Question 22

Correct

Mark 1 out of 1

Before you can use the interpolate() method to smooth the outliers in a DataFrame, you need to

- ☐ a. back fill the outliers
- ☐ b. forward fill the outliers
- ☒ c. change the outliers to missing values ✓
- ☐ d. convert the outliers to **

Question 23

Incorrect

Mark 0 out of 1

A SettingWithCopyWarning occurs when the DataFrame that you are trying to modify

- ☒ a. may be a copy of a view of a DataFrame ❌
- ☐ b. may be a copy of another DataFrame
- ☐ c. may be a reference to another DataFrame
- ☐ d. isn't a reference to another DataFrame

Question 24

Correct

Mark 1 out of 1

You should use the merge() method when you want to

- ☐ a. combine two columns in a DataFrame
- ☐ b. combine two DataFrames based on their index
- ☐ c. add the rows of one DataFrame to another DataFrame
- ☒ d. combine two DataFrames based on the data in one or more columns ✔️

Question 25

Correct

Mark 1 out of 1

To find and drop duplicate rows from a DataFrame, you can use the

- ☒ a. duplicated() and drop_duplicates() methods ✔️
- ☐ b. find_duplicates() and drop_duplicates() methods
- ☐ c. drop_duplicates() method
- ☐ d. info() and drop_duplicates() methods

Question 26

Incorrect

Mark 0 out of 1

If you replace all the invalid values in what should be a Boolean column with either True or False,

- ☒ a. you can then use the to_bool() method to convert the column to the bool data type ❌
- ☐ b. the column will be automatically converted to the bool data type
- ☐ c. the column will be automatically converted to the object data type
- ☐ d. you can then use the as_type() method to convert the column to the object data type

Question 27

Correct

Mark 1 out of 1

To find and fix datetime columns that are imported with the object data type, you can use

- ☐ a. the Pandas select_otypes() method and the to_datetime() method of the DataFrame
- ☐ b. the select_otypes() method of the DataFrame and the Pandas to_datetime() method
- ☐ c. the Pandas select_dtypes() method and the to_datetime() method of the DataFrame
- ☒ d. the select_dtypes() method of the DataFrame and the Pandas to_datetime() method ✓

Question 28

Correct

Mark 1 out of 1

To rename the columns in a DataFrame, you can use the Pandas rename() method or you can use the Python replace() method to replace the names in this attribute of a DataFrame:

- ☐ a. names
- ☐ b. column_names
- ☒ c. columns ✓
- ☐ d. headers

Question 29

Incorrect

Mark 0 out of 1

When you clean the data in a DataFrame, you first examine the data to see what needs to be done. Then, to start the cleaning process, you

- ☐ a. drop rows and columns that aren't needed, fix the data types, and fix any outliers
- ☐ b. drop duplicate rows, drop rows and columns that aren't needed, and rename columns
- ☐ c. drop columns that aren't needed, rename columns, and fix the data types
- ☒ d. drop rows and columns that aren't needed, rename columns, and fix missing values ✗

[◀ Quiz - Getting Data to Analyse](#)[Introduction to Python ▶](#)